

Introduzione

Cosa è un MCU/uC

Un *microcontrollore* è un dispositivo simile a un computer ma di dimensioni e prestazioni molto inferiori. Come un pc possiede una unità di controllo e numerose periferiche ma un MCU *contiene* la CPU e numerose periferiche racchiuse in un singolo chip mentre nei pc sono generalmente *esterne*. Anche dal punto di vista delle prestazioni le differenze tra MCU e sono importanti, ad esempio la quantità di memoria e la frequenza di clock di un tipico pc superano quelle di un MCU di diversi ordini di grandezza.

Varianti

Sul mercato esistono innumerevoli varianti di microcontrollori che si distinguono in base a diversi parametri come l'ampiezza del bus e l'architettura.

Ampiezza del bus Indica la quantità di linee che costituiscono il *bus dei dati* e determina la dimensione delle parole che il MCU può gestire. I MCU più comuni sono quelli a *8-bit* e possono manipolare dati di dimensione pari a 1 byte. Per applicazioni che richiedono prestazioni computazionali maggiori sono pure disponibili MCU a 16 e a 32 bit. L'aumento dell'ampiezza del bus comporta ovviamente un equivalente aumento dei consumi energetici di un MCU.

Tipo di memoria Esistono MCU che posseggono una memoria integrata e altri progettati per essere collegati ad una memoria esterna:

- Internal Memory MCU: sono i dispositivi più comuni, posseggono le memorie RAM e ROM necessarie al funzionamento integrate nel chip
- External Memory MCU: sono dispositivi che non posseggono una memoria integrata e quindi necessitano di una memoria esterna

Set di istruzioni Ogni MCU è capace di realizzare un numero predefinito di funzioni basilari, ogni funzione specifica è associata ad un codice mnemonico che corrisponde ad una istruzione del codice assembly. L'insieme di tutte queste istruzioni è detto *set di istruzioni* o *instruction set*. Ecco qualche esempio generico di istruzioni comuni in molti MCU:

- ADD, SUB, MUL, DIV: operazioni aritmetiche
- AND, OR, XOR: operazioni bit a bit
- MOV: *spostamento* di dati
- BC, BNC, BZ: controllo di flusso

Ogni MCU è capace di eseguire esclusivamente le funzionalità definite nel proprio *instruction set*, qualsiasi altra funzionalità *complessa* deve essere realizzata combinando le istruzioni di base.

Ogni produttore definisce il set di istruzioni dei propri dispositivi in modi differenti ma si possono distinguere due famiglie fondamentali:

- RISC, Reduced Instruction Set Computers: posseggono un set di istruzioni *ristretto* alle sole funzionalità fondamentali
- CISC, Complex Instruction Set Computers: posseggono un grande numero di funzionalità sofisticate (hardware implemented)

Architettura della memoria Un'altra distinzione importante tra diverse MCU è la cosiddetta *architettura della memoria*. Questo termine si riferisce al modo in cui i programmi e i dati vengono memorizzati dal sistema. Le due architetture predominanti sono la *Von Neumann* e la *Harvard*. La prima utilizza una sola memoria per conservare il programma e i dati (i valori delle variabili). La seconda utilizza invece due memorie separate, una ROM per le istruzioni del programma e una RAM per i dati.

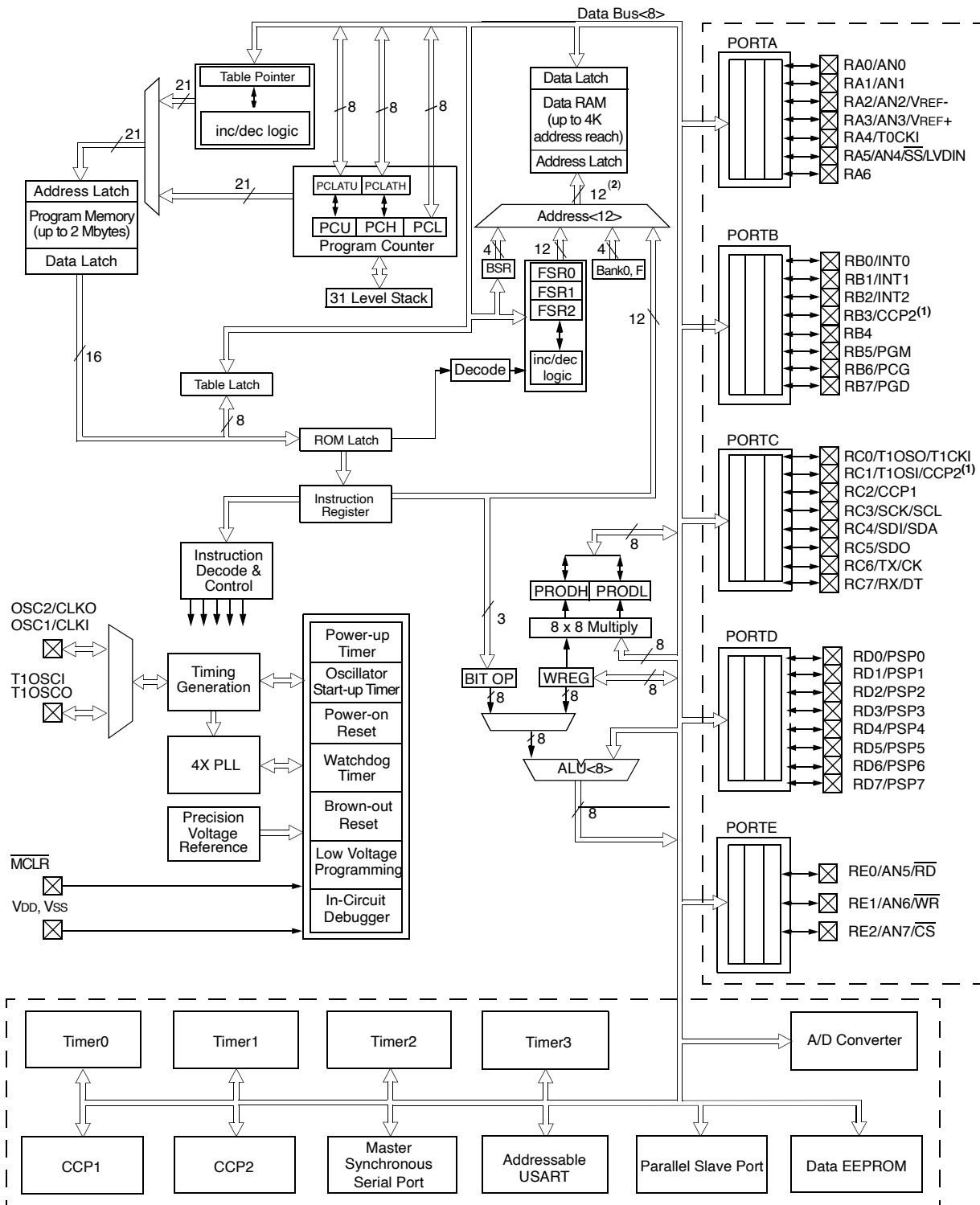


Generalmente l'architettura più comunemente utilizzata nell'ambito embedded è la *Harvard*.

Produttori/famiglie Ogni produttore di microcontrollori offre diverse *famiglie* di dispositivi che offrono differenti prestazioni. I prodotti più diffusi sono i PIC (Microchip) e gli ARM (Atmel).

Anatomia di un MCU

Un MCU contiene una CPU e tutte le periferiche necessarie al suo funzionamento in un singolo chip.



CPU È il *cervello* della MCU e esegue le istruzioni del programma. Contiene l'unità logica aritmetica (ALU) che serve a svolgere le operazioni. Un tipico MCU a 8 bit esegue una istruzione in 4 cicli di clock (detti *machine cycle*). Nelle macchine munite di *pipeline* (come il nostro PIC) l'esecuzione di una istruzione impiega apparentemente 1 ciclo.

RAM La *Random Access Memory* è l'unità che contiene i *registri* utilizzati per memorizzare i dati e per controllare il funzionamento di tutti gli altri moduli. I registri nella RAM si possono suddividere in due categorie: i GPR e i SFR.

- **GPR**, *General Purpose Registers*: sono utilizzati per memorizzare i valori delle variabili (allocate ad indirizzi casuali). La maggior parte dei registri nella RAM sono GPR.
- **SFR**, *Special Function Registers*: sono *registri speciali* e sono cablati (*hard-wired*) con la maggior parte dei moduli del MCU. Ciò consente di controllare ogni modulo scrivendo i valori del corrispondente SFR. Ad esempio, il TIMER0 può essere attivato/spento scrivendo 1 o 0 nel registro TMR0ON.

ROM La *Read Only Memory* serve a memorizzare le istruzioni del programma (assembly). Ogni singola istruzione è memorizzata in un registro della dimensione definita dall'architettura del MCU. Il numero di bit di un registro varia a dipendenza della MCU e del set di istruzioni. Il nostro PIC possiede una ROM con parole da 16 bit.

Le istruzioni del programma sono memorizzate nella ROM in ordine consecutivo e vengono indirizzate da uno specifico registro chiamato *program counter* (PC) che punta all'indirizzo dell'istruzione corrente (quella che deve essere eseguita). L'esecuzione di un intero programma consiste nell'incrementare il PC che definisce le istruzioni che andranno eseguite dal MCU.

Porte di IO Le porte di I/O sono la principale interfaccia tra il MCU e il *mondo esterno*. Tramite le porte di ingresso il MCU può leggere dei segnali digitali provenienti da componenti esterni, come pulsanti o sensori, e grazie alle uscite si possono pilotare dei dispositivi (es. led, driver motori).

Porte seriali Le porte seriali consentono al MCU di comunicare con altri dispositivi spedendo e ricevendo dei *flussi seriali* di dati. La comunicazione seriale consente di trasmettere qualsiasi tipo di informazione digitale, numeri, immagini, tracce audio e/o video o file.

Le porte seriali sono ampiamente utilizzate per creare delle interfacce macchina-macchina e uomo-macchina.

Tra le porte seriali di un MCU possono includere USART, I2C, SPI, USB e molti altri protocolli specifici a dipendenza dei campi di applicazione.

Timer Generalmente i MCU contengono un certo numero di *timer*. I timer possono essere utilizzati per temporizzare degli eventi e per misurare il tempo.

Interrupt Un *interrupt* è un evento che sospende l'esecuzione del programma principale mentre l'evento viene *gestito* da un altro programma. Attraverso l'uso di interrupt si riduce notevolmente il *tempo di reazione* del sistema ad eventi esterni. I MCU possono avere sorgenti di interrupt differenti, tra cui timer, porte seriali e perfino interrupt *software*.

Alla ricezione di un *segnale interrupt*, l'istruzione corrente viene sospesa, la sorgente dell'interruzione viene identificata e la CPU *salta* ad un indirizzo specifico della memoria in cui risiede una *interrupt service routine* (ISR). La ISR contiene il codice utile a gestire l'evento.

Gli interrupt sono controllati da un circuito logico che governa la generazione dei segnali di interrupt e comanda alla CPU di gestire le richieste di interruzione per tempo.

Oscillatore L'oscillatore è responsabile della sincronizzazione delle operazioni all'interno del MCU. Anche nei MCU privi di *oscillatore interno*, questo modulo è presente per generare un segnale di clock che sincronizza eventi e operazioni nel MCU.

Bus Sono le linee (i fili) che collegano tra loro tutte le parti del MCU. Un bus è definito per la sua *ampiezza*, ossia il numero di conduttori che lo compongono.

Periferiche Un MCU potrebbe avere alcune delle periferiche seguenti:

- EEPROM, Electrically Erasable Programmable Read Only Memory: per memorizzare dati in maniera *non volatile*.
- ADC, Analog To Digital Converter: per convertire una tensione analogica in un valore digitale.
- DAC, Digital To Analog Converter: per generare tensioni analogiche, es. forme d'onda.
- PWM, Pulse Width Modulation: per generare segnali di controllo PWM.
- Comparator: è un *comparatore analogico*, utile per confrontare due segnali analogici senza doverli prima convertire in digitale.

il **pic18** Nel nostro corso utilizzeremo un PIC18f45k22 che un MCU 8 bit ad alte prestazioni:

- 8-bit
- memoria interna
- RISC
- architettura Harvard

Features	PIC18F23K22 PIC18LF23K22	PIC18F24K22 PIC18LF24K22	PIC18F25K22 PIC18LF25K22	PIC18F26K22 PIC18LF26K22	PIC18F43K22 PIC18LF43K22	PIC18F44K22 PIC18LF44K22	PIC18F45K22 PIC18LF45K22	PIC18F46K22 PIC18LF46K22
Program Memory (Bytes)	8192	16384	32768	65536	8192	16384	32768	65536
Program Memory (Instructions)	4096	8192	16384	32768	4096	8192	16384	32768
Data Memory (Bytes)	512	768	1536	3896	512	768	1536	3896
Data EEPROM Memory (Bytes)	256	256	256	1024	256	256	256	1024
I/O Ports	A, B, C, E ⁽¹⁾	A, B, C, E ⁽¹⁾	A, B, C, E ⁽¹⁾	A, B, C, E ⁽¹⁾	A, B, C, D, E	A, B, C, D, E	A, B, C, D, E	A, B, C, D, E
Capture/Compare/PWM Modules (CCP)	2	2	2	2	2	2	2	2
Enhanced CCP Modules (ECCP) - Half Bridge	2	2	2	2	1	1	1	1
Enhanced CCP Modules (ECCP) - Full Bridge	1	1	1	1	2	2	2	2
10-bit Analog-to-Digital Module (ADC)	2 internal 17 input	2 internal 17 input	2 internal 17 input	2 internal 17 input	2 internal 28 input	2 internal 28 input	2 internal 28 input	2 internal 28 input
Packages	28-pin PDIP 28-pin SOIC 28-pin SSOP 28-pin QFN 28-pin UQFN	28-pin PDIP 28-pin SOIC 28-pin SSOP 28-pin QFN 28-pin UQFN	28-pin PDIP 28-pin SOIC 28-pin SSOP 28-pin QFN	28-pin PDIP 28-pin SOIC 28-pin SSOP 28-pin QFN	40-pin PDIP 40-pin UQFN 44-pin QFN 44-pin TQFP	40-pin PDIP 40-pin UQFN 44-pin QFN 44-pin TQFP	40-pin PDIP 40-pin UQFN 44-pin QFN 44-pin TQFP	40-pin PDIP 40-pin UQFN 44-pin QFN 44-pin TQFP
Interrupt Sources	33							
Timers (16-bit)	4							
Serial Communications	2 MSSP, 2 USART							
SR Latch	Yes							
Charge Time Measurement Unit Module (CTMU)	Yes							
Programmable High/Low-Voltage Detect (HLVD)	Yes							
Programmable Brown-out Reset (BOR)	Yes							
Resets (and Delays)	POR, BOR, RESET Instruction, Stack Overflow, Stack Underflow, (PWR1, OST), MCLR, WDT							
Instruction Set	75 Instructions; 83 with Extended Instruction Set enabled							
Operating Frequency	DC - 64 MHz							

Primo programma

L'IDE che utilizzeremo per sviluppare i programmi per PIC si chiama **MPLAB X IDE** ed è distribuito gratuitamente da Microchip.

Creare un progetto

Per creare un nuovo programma con **MPLAB X** bisogna innanzitutto creare un nuovo progetto:

1. Premere il tasto **New Project**
2. Scegliere tipo di progetto: selezionare la categoria **Microchip Embedded** e il tipo di progetto **Application Project(s)**
3. Selezionare il modello di MCU: scegliere nel campo **Device** il codice del nostro micro **PIC18F45K22** e nel campo **Tool** il modello del loader/debugger che si utilizza (**PICKit 4**, **ICD4**, ...)
4. Scegliere il compilatore: selezionare il compilatore **XC8**
5. Scegliere nome e percorso del progetto
6. Togliere la spunta **"Open MCC on Finish"**

Al termine del procedimento l'IDE *crea* il progetto che consiste in una cartella detta *cartella di progetto* che contiene una struttura predefinita di cartelle e file.

Il nuovo progetto viene aperto e compare nel pannello **Projects** dove viene rappresentato come una gerarchia di cartelle e file. Per il momento ci interessa solamente la cartella **Source Files** che è predisposta per contenere il *codice sorgente*.

Creare `main.c`

Ora possiamo creare il file `main.c` in cui scriveremo il codice del programma:

1. Nel pannello **Projects** selezionare la cartella **Source Files** e premere il tasto destro del mouse
2. Nel menu a tendina selezionare **New>main.c**
3. Fornire il nome del file da creare (**main**)

Creare `config.h`

Prima di procedere con la stesura del codice del programma vero e proprio, bisogna creare un *header file* (un file `.h`) contenente la configurazione del PIC:

1. Nel pannello **Projects** selezionare la cartella **Header Files** e premere il tasto destro del mouse
2. Nel menu a tendina selezionare **New>xc8_header.h**
3. Fornire il nome del file da creare (**config**)

MPLAB X contiene un tool che facilita la stesura del file di configurazione, per aprirlo selezionare nel menu **Window>Target Memory Views>Configuration Bits**.

Approfondiremo meglio i dettagli della configurazione in seguito, per il momento adotteremo la configurazione di default con solo due cambiamenti:

1. Oscillatore esterno **FOSC**: useremo l'oscillatore esterno da 16MHz (**HSHP**)
2. Abilitazione del *watch dog* (**WDTEN**): il *watch dog* deve essere disabilitato (**OFF**)
3. Abilitazione A/D su Porta B (**PBADEN**): useremo i pin della porta B come ingressi digitali, quindi la conversione A/D sulla porta B deve essere disabilitata (**OFF**)

Tramite l'interfaccia del tool **Configuration Bits** è possibile visualizzare e modificare il valore di tutti i bit di configurazione e poi generare il codice del corrispondente *header file*:

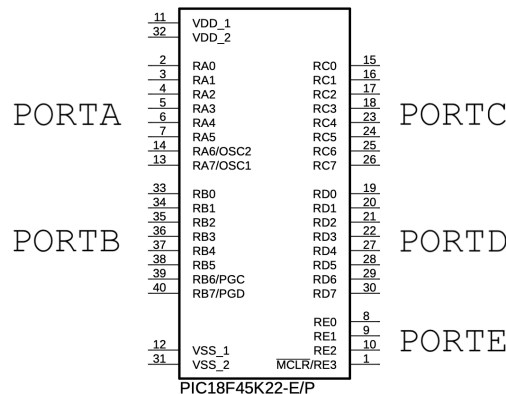
1. Aprire il tool **Configuration Bits** (**Window>Target Memory Views>Configuration Bits**)
2. Modificare in modo appropriato i valori dei bit di configurazione (**FOSC=HSHP**, **WDTEN=OFF** e **PBADEN = OFF**)
3. Generare il codice: premere il tasto **Generate Source Code to Output**
4. Selezionare e copiare il codice generato
5. Incollare il codice copiato nel file `config.h` sostituendo tutto il contenuto originale
6. Nel file `main.c` *includere* in file `config.h`:

```
#include "config.h"
```

```
void main(void) {...}
```

Configurare le porte GPIO

I *General Purpose Input/Output pins* sono pin configurabili via software per essere utilizzati come ingressi o come uscite. Gran parte dei pin del PIC18F45K22 sono GPIO:



I pin GPIO del PIC18F45K22 sono suddivisi in 5 *porte*:

- PORTA: possiede 7 pin, da RA0 a RA6 (RA6 *occupato* come OSC2)
- PORTB: possiede 8 pin, da RB0 a RB7
- PORTC: 8 pin, da RC0 a RC7
- PORTD: 8 pin, da RD0 a RD7
- PORTE: 3 pin, da RE0 a RE2

I pochi pin restanti sono:

- Pin di alimentazione (VDD_1, VDD_2, VSS_1, VSS_2)
- Pin dell'oscillatore (OSC1 e OSC2)
- Pin di reset (MCLR)

Configurare un pin come input o output

Il funzionamento di ogni *porta* GPIO del PIC è regolato da tre registri:

- TRIS (*data direction register*): ogni bit del registro definisce la *direzione* del corrispondente pin, ossia se il pin è configurato come input (1) o come output (0)
- PORT: ogni bit del registro contiene il valore logico attuale sul corrispondente pin
- LAT: rappresenta l'*output latch*, ossia lo stato del latch di output di ogni pin (l'ultimo stato scritto)

Ogni porta possiede i propri 3 registri identificati dalla corrispondente lettera. Ad esempio i registri TRISA, PORTA e LATA sono associati alla porta A, TRISB, PORTB e LATB alla porta B, e via dicendo.

A titolo di esempio proviamo a configurare alcuni GPIO. Sulla nostra scheda hardware sono presenti 4 pulsanti (da S1 a S4) e 4 diodi led (da LD1 a LD4). I pulsanti sono collegati ai pin da RB0 a RB3 e i led sono collegati ai pin da RB4 a RB7.

Per poter *leggere* lo stato dei pulsanti e pilotare i led dobbiamo innanzitutto configurare correttamente i pin GPIO della porta B. S1 è collegato al pin RB0, che deve quindi essere configurato come input. Per configurare un pin come ingresso è necessario *settare* un bit (scrivere un 1) nel corrispondente registro TRIS. I registri sono strutturati nel modo seguente:

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
PORTB	RB7	RB6	RB5	RB4	RB3	RB2	RB1	RB0
LATB	LATB Data Output Register							
TRISB	PORTB Data Direction Register							

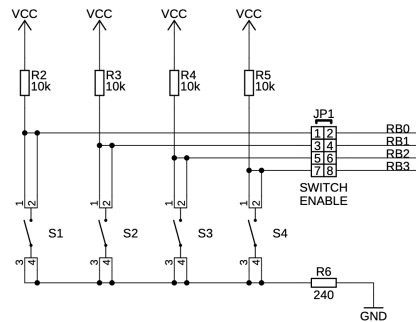
Il pin RB0 è associato al *bit meno significativo* dei registri, mentre RB7 al *bit più significativo*. Per configurare il RB0 bisogna quindi scrivere un 1 nel lsb di TRISB. Ciò è possibile in diversi modi, il più semplice consiste nell'uso della costante simbolica TRISBbits.TRISB0:

```
TRISBbits.TRISB0 = 1; // set RB0 as input
```

In alternativa è pure possibile scrivere un byte nel registro TRISB, ciò consente di configurare tutti i pin in una sola istruzione. Ad esempio per configurare le porte da RB0 a RB3 come input e le restanti (RB4 a RB7) come output, si può scrivere il valore 0x0F nel registro:

```
TRISB = 0x0F; // set RB0-RB3 as input, RB4-RB7 as output
```

Lettura di un pin di input



Il pulsante S1 è collegato ad una *resistenza di pull-up*, di conseguenza:

- quando S1 è aperto, lo stato logico del pin RB0 è alto
- quando S1 è chiuso, lo stato logico del pin RB0 è basso

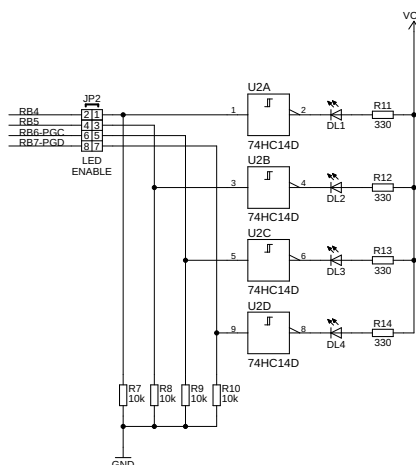
Il valore logico del pin può essere letto nel modo seguente:

```
if(PORTBbits.RB0 == 1){
    ...
}
```

O più semplicemente:

```
if(PORTBbits.RB0){
    ...
}
```

Scrittura di un pin di output



Il led LD1 è pilotato dal pin LB4 (il bit 4 del registro LATB) attraverso uno Schmitt trigger invertente, di conseguenza:

- quando LB4 è alto, il LD1 è acceso

- quando LB4 è basso, il LD1 è spento

Per cambiare lo stato di un pin di output è sufficiente settare il corrispondente bit nel registro LAT (o PORT). Ad esempio per accendere LB:

```
LATBbits.LB4 = 1; // Turn LD1 ON
```

Per spegnere LB4:

```
LATBbits.LB4 = 0; // Turn LD1 OFF
```

Ora possiamo leggere lo stato di S1 per pilotare il led LD1:

```
while (1) {  
    if (PORTBbits.RB0) {  
        LATBbits.LB4 = 1;  
    } else {  
        LATBbits.LB4 = 0;  
    }  
}
```

Oppure più semplicemente:

```
while (1) {  
    LATBbits.LB4 = PORTBbits.RB0;  
}
```

In questo modo il led è acceso quando S1 è aperto, per invertire il comportamento è sufficiente usare una negazione:

```
while (1) {  
    LATBbits.LB4 = !PORTBbits.RB0;  
}
```


Configurazione dei pin GPIO

Abbiamo appreso che per manipolare la *direzione* (I/O) di un pin GPIO è necessario modificare il rispettivo bit in un registro TRISx (dove x sta per A, B, C, D o E). La scrittura di un 1 configura il pin come ingresso, mentre uno 0 indica che il pin verrà utilizzato come uscita.

Tutti i bit dei registri TRISx sono *mappati* tramite una **union** che consente di accedere direttamente ai singoli bit dei registri (detti *bit fields*). Le **union** sono definite nell'header file specifico del MCU incluso automaticamente da MPLAB X (es. `pic18f45k22.h`) e possono essere visualizzati tramite la funzione di menu **Go to declaration**. Ad esempio per TRISB viene definita la costante simbolica `TRISBbits` nel modo seguente:

```
typedef union {
    struct {
        unsigned TRISB0      :1;
        unsigned TRISB1      :1;
        unsigned TRISB2      :1;
        unsigned TRISB3      :1;
        unsigned TRISB4      :1;
        unsigned TRISB5      :1;
        unsigned TRISB6      :1;
        unsigned TRISB7      :1;
    };
    struct {
        unsigned RB0         :1;
        unsigned RB1         :1;
        unsigned RB2         :1;
        unsigned RB3         :1;
        unsigned RB4         :1;
        unsigned RB5         :1;
        unsigned RB6         :1;
        unsigned RB7         :1;
    };
} TRISBbits_t;
extern volatile TRISBbits_t TRISBbits __at(0xF93);
```

Grazie a `TRISBbits` è riferirsi ad ogni bit del registro in due modi, ad esempio per il primo bit meno significativo:

```
TRISBbits.RB0 = 1; //RB0 input
```

Oppure:

```
TRISBbits.TRISB4 = 0; //RB4 output
```

Inoltre vengono pure definite le costanti utili ad accedere direttamente ai singoli bit (senza fare riferimento a `TRISBbits`), ad esempio `TRISB0`:

```
extern volatile __bit TRISB0 __at(0x7C98); // @ (0xF93 * 8 + 0)
#define TRISB0_bit _BIT_ACCESS(TRISB,0)
```

Ciò consente di semplificare la sintassi precedente accedendo direttamente al bit desiderato:

```
TRISB0 = 1; //RB0 input
TRISB1 = 1; //RB1 input
TRISB2 = 1; //RB2 input
TRISB3 = 1; //RB3 input
TRISB4 = 0; //RB4 output
TRISB5 = 0; //RB5 output
TRISB6 = 0; //RB6 output
TRISB7 = 0; //RB7 output
```

Overwrite di registri:

I registri `TRISx`, `PORTx` e `LATx` possono però essere modificati pure *in blocco*. È infatti possibile scrivere un intero byte in un registro, i valori degli 8 bit verranno sovrascritti interamente. Ad esempio per configurare i pin della porta B abbiamo scritto:

```
TRISB = 0x0F; //RB7-RB4 output, RB3-RB0 input
```

Oppure:

```
TRISB = 0b00001111; //RB7-RB4 output, RB3-RB0 input
```

Lo stesso vale pure per i registri PORTx e LATx:

```
LATB = 0xFF; //RB0-RB7 tutti ON
LATB = 0;    //RB0-RB7 tutti OFF
```

La sovrascrittura di un registro è semplice e diretta ma in alcune circostanze non è adeguata perché sovrascrive tutti i bit nel registro. Quando si vogliono modificare alcuni bit di un registro lasciando però inalterati gli altri, bisogna ricorrere all'uso dei *bit fields* mostrati precedentemente oppure al cosiddetto *bit masking*.

Bit masking

Resettare uno o più bit La tecnica del *bit masking* permette di modificare alcuni bit specifici di un registro senza modificare il valore degli altri bit. Ad esempio se volessimo spegnere i pin RB4, RB5, RB6 e RB7 senza modificare lo stato degli altri pin del registro PORTB potremmo fare nel modo seguente:

```
LATB = LATB & 0x0F; // spegne RB4-RB7, lascia tutti gli altri pin inalterati
```

O anche:

```
LATB &= 0x0F; // spegne RB4-RB7, lascia tutti gli altri pin inalterati
```

Per spegnere solo i pin RB4 e RB6 potremmo invece scrivere:

```
LATB = LATB & 0xAF; // spegne RB4 e RB6, lascia tutti gli altri pin inalterati
```

Settare uno o più bit Per accendere i pin RB4 e RB6 potremmo invece utilizzare l'operatore OR:

```
LATB = LATB | 0x50; // accende RB4 RB6, lascia tutti gli altri pin inalterati
```

O anche:

```
LATB |= 0x50; // accende RB4 RB6, lascia tutti gli altri pin inalterati
```

Toggle di uno o più bit È pure possibile utilizzare gli *operatori bit a bit* per invertire lo stati di uno o più bit di un registro (toggle). Ciò è facilmente realizzabile tramite l'operatore XOR. Ad esempio per *invertire* lo stato del bit RB6 senza toccare gli altri 7:

```
LATB = LATB ^ 0x40;
```

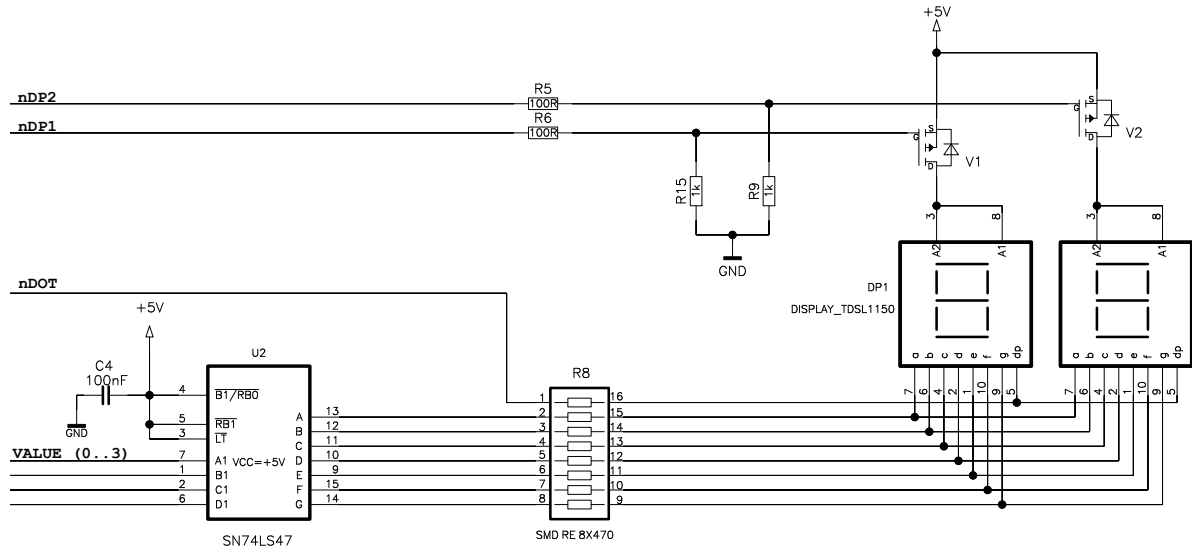
Oppure:

```
LATB ^= 0x40;
```

Lettura di un bit Il *bit masking* può essere utilizzato pure per *leggere* lo stato di un singolo bit da un registro, ad esempio per leggere lo stato di RB1:

```
if(PORTB & 0x2)
```

Uso dei display



I due display della scheda BAB sono pilotati attraverso un *decoder 7 segmenti* SN74LS47 tramite la porta PORTD:

7	6	5	4	3	2	1	0
NU	nDP1	nDP2	nDOT	VALUE			

Dove:

- nDP2 (RD6): attiva il display 2 (il prefisso n indica negazione, quindi il display è attivo quando il bit è 0)
- nDP1 (RD5): attiva il display 1
- nDOT (RD4): accende il punto (dot) nel display attivo
- VALUE (RD3..0): nibble che rappresenta il valore da rappresentare sul display attivo (vedi datasheet SN74LS47)

Tipi di dati

Tipi interi

Il compilatore MPLAB XC8 supporta tipi interi da 1, 2, 4 e 8 byte:

Type	Size (bits)	Arithmetic Type
signed char	8	Signed integer
unsigned char	8	Unsigned integer
signed short	16	Signed integer
unsigned short	16	Unsigned integer
signed int	16	Signed integer
unsigned int	16	Unsigned integer
signed long	32	Signed integer
unsigned long	32	Unsigned integer
signed long long	64	Signed integer
unsigned long long	64	Unsigned integer

Se la marcatura **signed/unsigned** è omessa, il tipo è ogni tipo intero è per default *signed* (inclusi i **char**). Gli interi **signed** sono memorizzati in *complemento a due*.

Gli intervalli di valori che possono essere rappresentati da questi tipi di dati sono raccolti nell'header file `<limits.h>` che può essere incluso nei file sorgente in cui si desidera renderli accessibili.

Per rendere più esplicite le caratteristiche dei tipi di dati, è preferibile utilizzare i tipi a *campo fisso* (*fixed-width types*¹) che posseggono una dimensione fissa (non *platform dependent*) e dei nomi più espressivi. Per il PIC18 questi tipi sono definiti in `alltypes.h`:

Type	Size (bits)	Arithmetic Type
int8_t	8	Signed integer
uint8_t	8	Unsigned integer
int16_t	16	Signed integer
uint16_t	16	Unsigned integer
int24_t	24	Signed integer
uint24_t	24	Unsigned integer
int32_t	32	Signed integer
uint32_t	32	Unsigned integer
int64_t	64	Signed integer
uint64_t	64	Unsigned integer

Gli intervalli rappresentabili per i *fixed-width types* si trovano in `stdint.h`.

Tipi a virgola mobile

XC8 supporta esclusivamente numeri a virgola mobile da 32 bit associati ai tipi `float`, `double` e `long double` che sono sinonimi.

¹Vedi *Integer Data Types*.